

Group Project: Library Management System

Database Systems Course

Group Size: 4 Students

Due: Presentation on Last Lesson

1 Project Overview

This group project involves designing and implementing a **Library Management System** using MySQL in a Docker environment and a Python Flask web application to interact with the database. The project integrates concepts from the course, including database setup, data independence, DBMS architecture, relational and EER modeling, SQL queries, DDL, and normalization. Your team of four students will collaborate to design the database, develop the application, and present your work during the last lesson.

2 Objectives

- Apply database concepts from Labs 1–10 to design and implement a normalized relational database.
- Develop a Flask web application to perform CRUD (Create, Read, Update, Delete) operations on the database.
- Demonstrate data independence, query optimization, and EER modeling.
- Present the project, showcasing the database design, application functionality, and lessons learned.

3 Tools Required

- Docker Desktop (or Docker CLI)
- MySQL client (e.g., MySQL Workbench or CLI)
- Python 3, Flask (`pip install flask`), MySQL Connector (`pip install mysql-connector-python`)

4 Prerequisites

- Completion of Labs 1–10.
- Understanding of SQL (DDL, DML, queries), ER/EER modeling, normalization, and Flask integration.

5 Project Requirements

5.1 Database Design

- **ER/EER Modeling** (Labs 8, 9): Design an EER model for the library system, including entities (Books, Borrowers, Loans) and a specialization (e.g., Books with subtypes Textbooks and Novels).
- **Normalization** (Lab 10): Normalize the database to 3NF to eliminate redundancy.
- **DDL Implementation** (Lab 7): Use DDL to create tables with appropriate constraints (primary keys, foreign keys, NOT NULL, CHECK).
- **Data Independence** (Lab 2): Create views to demonstrate logical data independence.

5.2 Flask Application

- **Integration** (Lab 8): Develop a Flask application to interact with the MySQL database.
- **CRUD Operations**: Implement endpoints for:
 - Creating a new book or borrower.
 - Reading book or loan details (using JOIN queries from Lab 6).
 - Updating book availability or loan status.
 - Deleting a loan record.
- **Query Optimization** (Labs 5, 6): Use indexes to optimize queries and analyze performance with EXPLAIN (Lab 3).

5.3 Presentation

- Prepare a 10-minute presentation for the last lesson.
- Include:
 - EER diagram and explanation of normalization.
 - Demonstration of the Flask application (live or via screenshots).
 - Discussion of challenges faced and how course concepts were applied.

6 Team Roles

Each team member should take primary responsibility for one of the following roles, with all members contributing to all aspects:

1. **Database Designer**: Leads ER/EER modeling, normalization, and DDL implementation.

2. **Backend Developer:** Implements the Flask application and database connectivity.
3. **Query Specialist:** Writes and optimizes SQL queries and views.
4. **Presentation Coordinator:** Organizes the presentation and ensures all deliverables are prepared.

7 Deliverables

- **Database:** SQL scripts for table creation, sample data, views, and indexes.
- **Flask App:** Source code (`app.py`) with at least three CRUD endpoints.
- **EER Diagram:** Visual representation of the database design.
- **Presentation:** 10-minute slide deck and live demo (or screenshots).
- **Report:** Brief document (1–2 pages) summarizing the project, roles, and lessons learned.

8 Evaluation Criteria

- **Database Design (50%):** Correct EER modeling, normalization, and DDL implementation.
- **Application Functionality (20%):** Working Flask app with CRUD operations and optimized queries.
- **Presentation (20%):** Clarity, coverage of course concepts, and team collaboration.
- **Report (10%):** Quality of documentation and reflection on challenges.